Copyright © 2025 Department of Systems and Computer Engineering, Carleton University

SYSC 2004 – Object-Oriented Software Development

Object Oriented Concepts

UML Class Diagrams

Refactoring

Carleton University

Lecture 3 Summary

- UML notation: compartments: name, attributes, operations, *responsibilities*
- Java class syntax: fields, constructors, methods
- Creating Java objects: BlueJ's Object bench, inspect

Lecture 4 Objectives

By the end of this lecture, you should be able to:

- Understand basic object-oriented programming concepts
  - Encapsulation and Data Hiding
  - Interface and Implementation
  - Inheritance
  - Polymorphism
  - Composition
- Represent these concepts using UML class diagrams
- Refactoring

Encapsulation and Data Hiding

IntSquare
- squareValue: int
+ getSquare(value: int): int - calculateSquare(value: int): int

- Encapsulation
  - Attributes and behaviors are contained within the same entity
- Data hiding
  - Some elements are hidden from other objects

Interfaces and Implementations are the basics of encapsulation

Not to be confused with Java Interfaces or with “program against an interface”.
We will talk about these later in the course

Interfaces and Implementations

IntSquare
- squareValue: int
+ getSquare(value: int): int - calculateSquare(value: int): int

Interface: Means of communicating between objects.

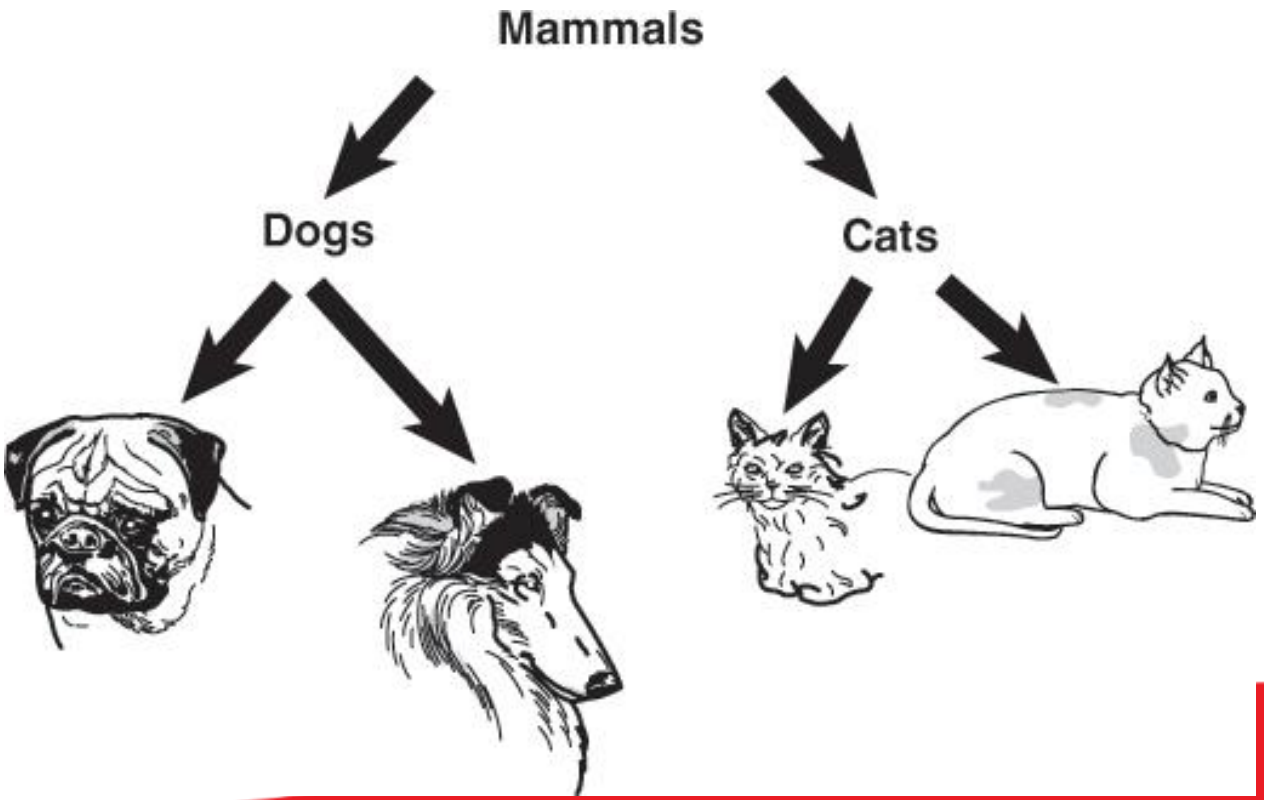
Implementation: How things are coded, e.g. algorithm used.

```
public int getSquare (int value){
  squareValue = calculateSquare(value);
  return squareValue;
}
```

The same interface can have different implementations:
The functionality does not change

Inheritance

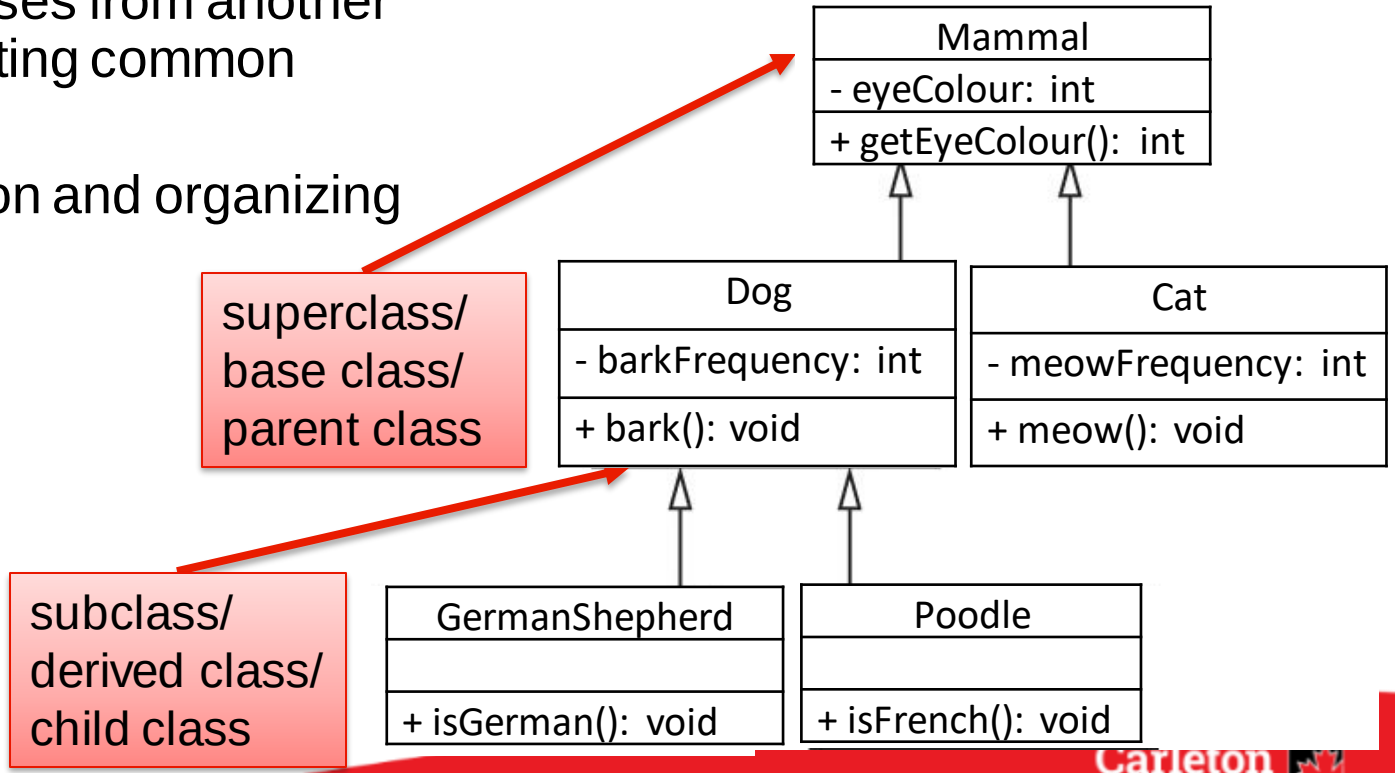
IS-A RELATIONSHIP



Inheritance

IS-A RELATIONSHIP

- Create new classes from another class by abstracting common attributes
- Allows abstraction and organizing concepts



Inheritance

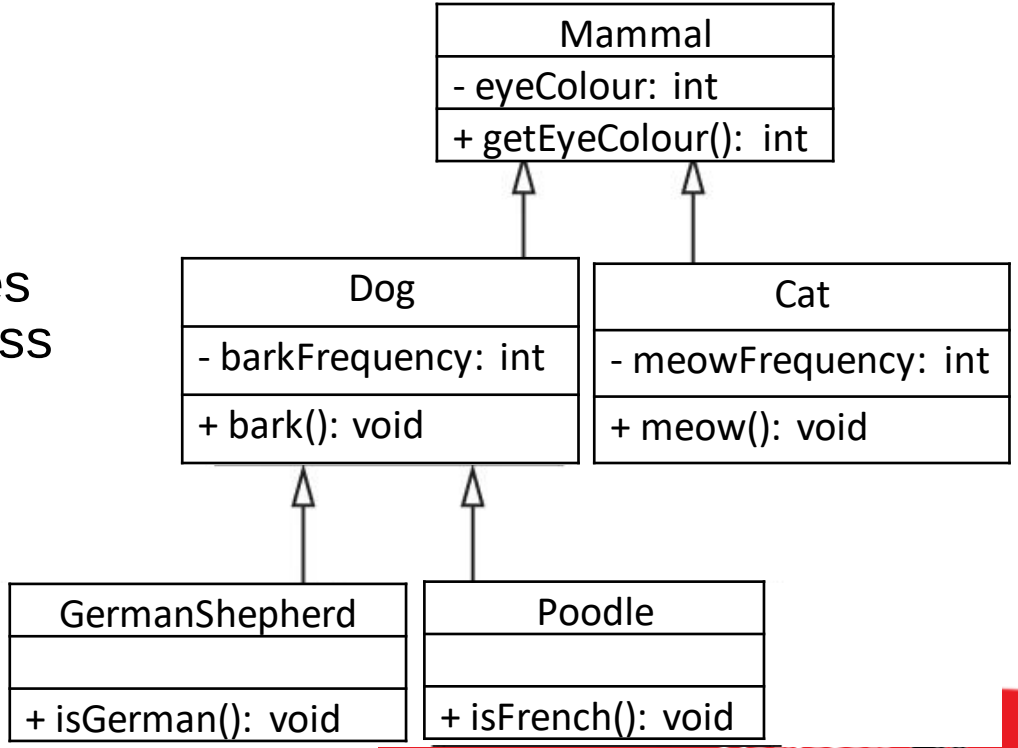
IS-A RELATIONSHIP

Remarks about UML

- Note the arrows to represent inheritance
- Inherited methods and attributes are NOT included in the subclass

In Java:

- `extends` keyword



Note: This is just an introduction. More about this later in the course.

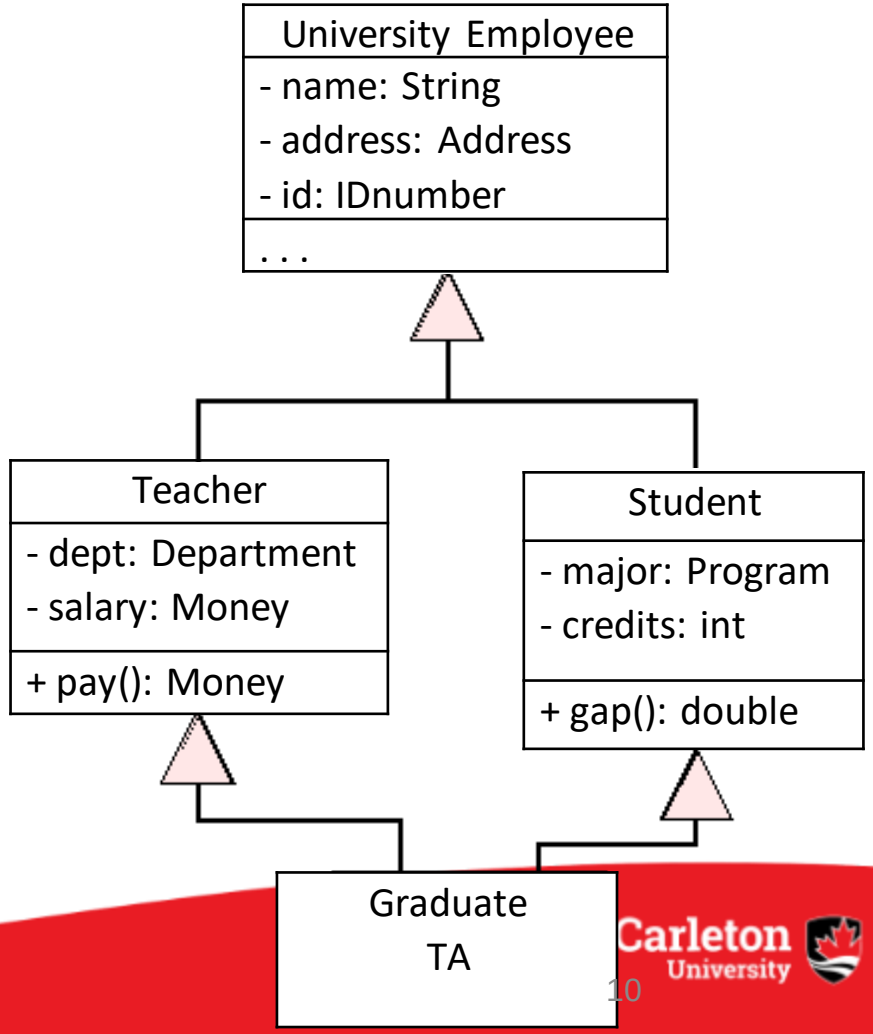
Inheritance

IS-A RELATIONSHIP

- It is difficult to decide how much abstraction is needed
- Current behavior by developers: Inheritance is mainly used for attributes

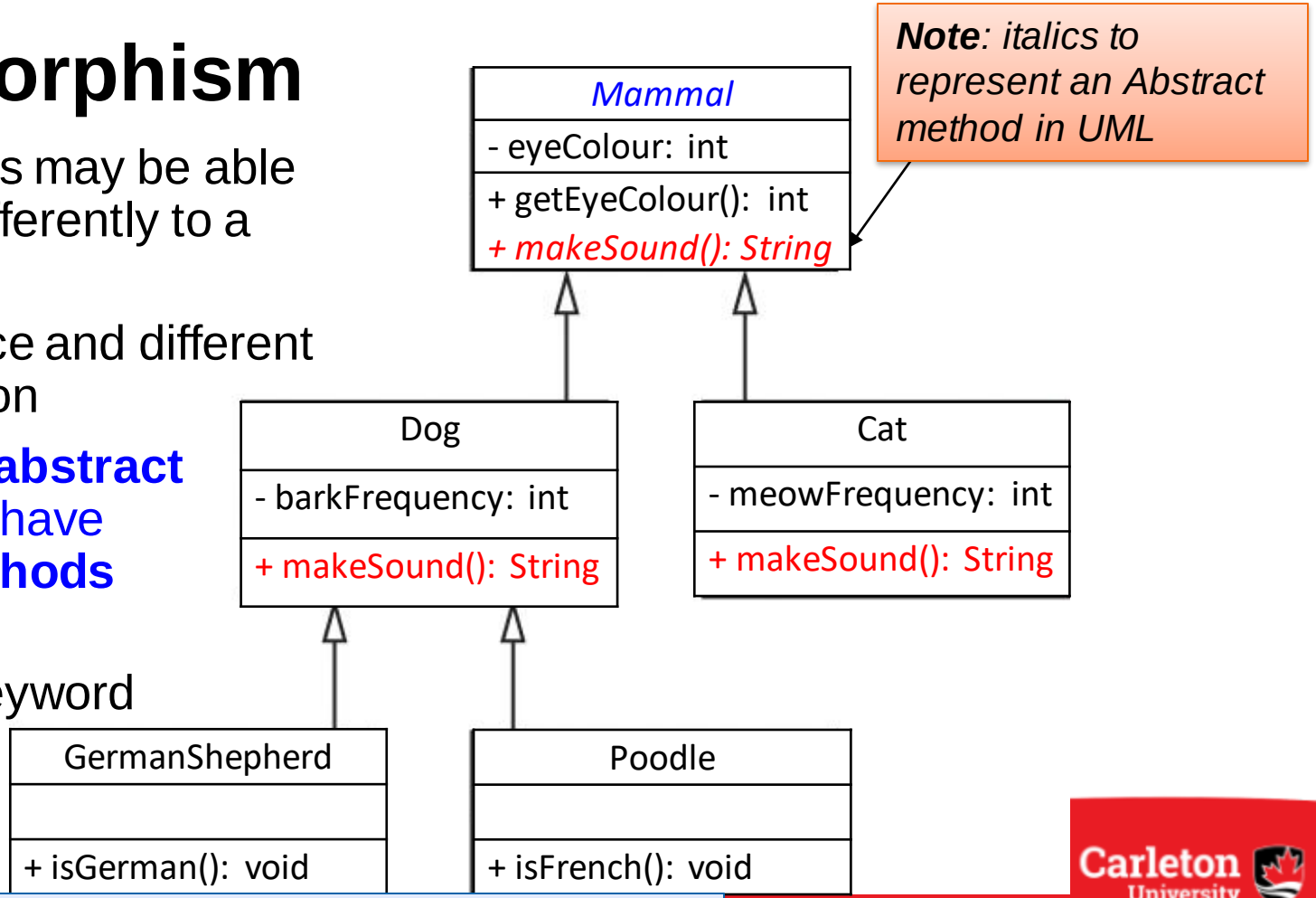
Multiple Inheritance

- Multiple Inheritance is a problem when writing compilers
- Java does not support multiple inheritance (see shorturl.at/fkJP7)
- C++ supports Multiple Inheritance



Polymorphism

- Each subclass may be able to respond differently to a method
- Same interface and different implementation
- In Java, only **abstract classes** may have **abstract methods**
- **In Java:** `abstract` keyword

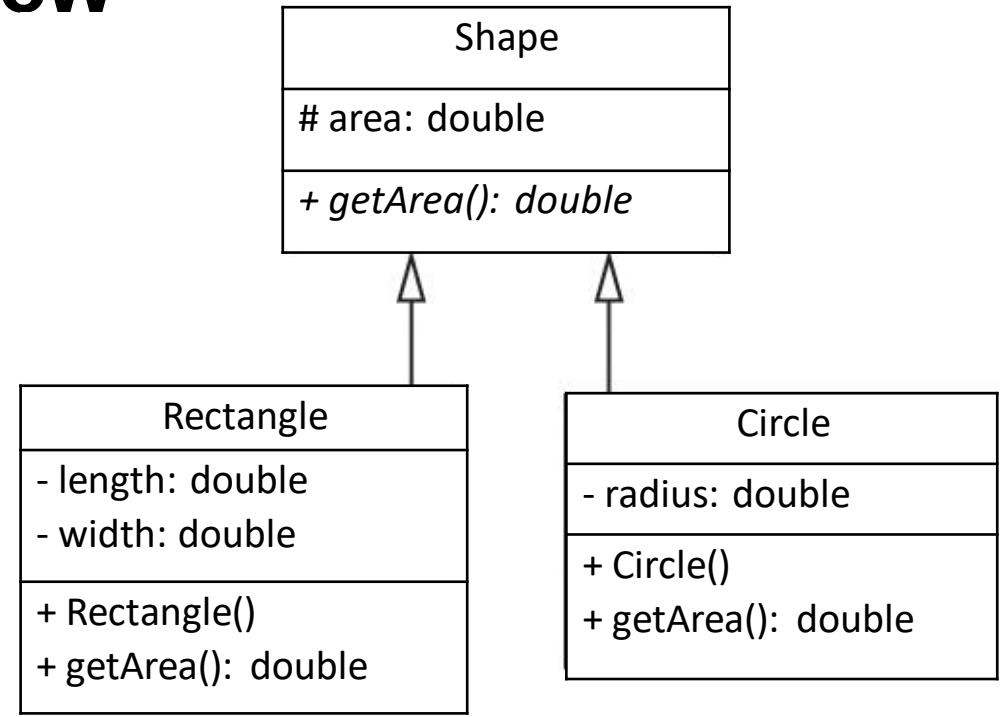


Note: This is just an introduction. More about this later in the course.

UML Notation review

- 3 types of visibility:
  - private
  + public
  # protected

  New symbol
- Shape of the arrows for inheritance:
  In Java `extends`
- Italics for abstract method. A class with an abstract method is called abstract class:
  In Java `abstract`



Composition

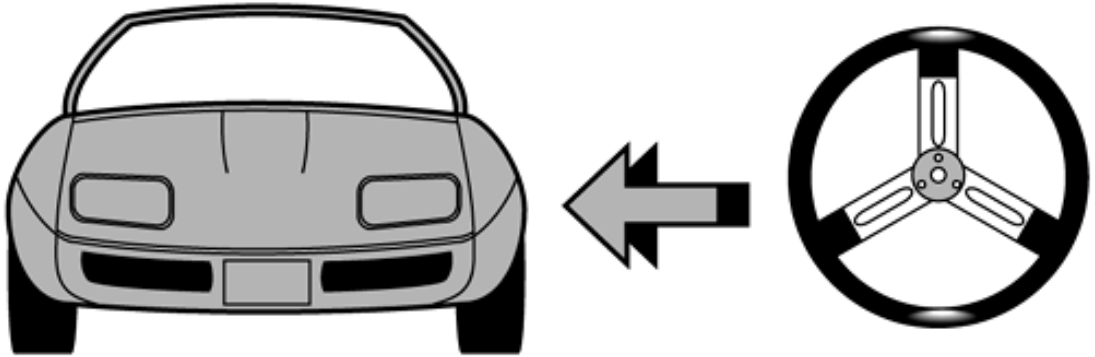
HAS-A RELATIONSHIP

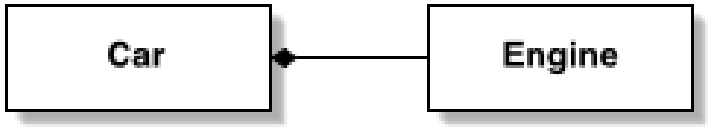
Mechanism for building objects from other objects

- Composition
- Aggregation

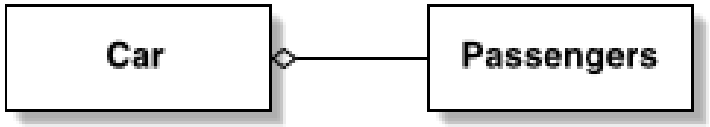
Note the UML "arrows"

A Car has a Steering Wheel





Composition: every car has an engine.



Aggregation: cars may have passengers, they come and go

Note: This is just an introduction. More about this later in the course.

REFACTORING: PEN EXAMPLE

Carleton University

Class `Pen`: First Version of `drawSquare()` (Pen-v2a)

Draws a 50-by-50 pixel square, starting at the upper-left corner

Let us take a look at the code in Pen-v2a

What is wrong with this implementation?

Size of square is *hardwired* into the method

Class `Pen`: Second Version of `drawSquare()` (Pen-v2b)

Try again: pass the length of the square's sides as a parameter

Let us take a look at the code in Pen-v2b

Did the interface change? Did the implementation change?

Interface – Yes: from no input parameter to one integer
Implementation – Yes

What is wrong with this implementation?

It is simpler to use Pen methods rather than Canvas methods

Class `Pen`: Third Version of `drawSquare()` (Pen-v2c)

Try again: use `drawLineOnCanvas()`, not `Canvas.drawingCanvas.drawLine()`

Let us take a look at the code in Pen-v2c

Did the interface change? Did the implementation change?

Interface – No: same method signature
Implementation – Yes

What is wrong with this implementation?

The method works, but has smelly code

Class `Pen`: Third Version of `drawSquare()` (Pen-v2c)

What is wrong with this implementation?
The method works, but has smelly code

Code smell: a hint that something may be wrong with the code (but it might be ok and not need improvement)

drawSquare() algorithm:

```
set pen direction to EAST
draw horizontal line
set pen direction to SOUTH
draw vertical line
set pen direction to WEST
draw horizontal line
set pen direction to NORTH
draw vertical line
```

Pen has methods to change the pen's direction and draw a line, but `drawSquare()` duplicates what they do.

Refactoring

- "Refactoring is a disciplined technique for restructuring an existing body of code, altering its internal structure without changing its external behavior. Its heart is a series of small behavior preserving transformations. Each transformation (called a 'refactoring') does little, but a sequence of transformations can produce a significant restructuring. Since each refactoring is small, it's less likely to go wrong. The system is also kept fully working after each small refactoring, reducing the chances that a system can get seriously broken during the restructuring.“
  - http://www.refactoring.com
- Goal: code that is shorter, cleaner, simpler, and is therefore easier to understand and modify

Class `Pen`: Fourth Version of `drawSquare()` (Pen-v2d)

Try again: Replace chunks of code by calls to `moveBy()` and `turnClockwise()`

Let us take a look at the code in Pen-v2d

Did the interface change? Did the implementation change?

Interface – No: same method signature
Implementation – Yes

Class `Pen`: Fifth Version. add `drawRectangle()` (Pen-v2e)

Let us take a look at the code in Pen-v2e

Class `Pen`: Six Version of `drawSquare()` (Pen-v2f)

- Notice the similarity between `drawRectangle()` and `drawSquare()`
- `drawRectangle()` is more general than `drawSquare()`:
  - `drawSquare()` can be viewed as a special case of `drawRectangle()`
- Refactor `drawSquare()` again (replace 10 lines of code by one line)

Let us take a look at the code in Pen-v2f

Did the interface change? Did the implementation change?

Interface – No: same method signature
Implementation – Yes

Learning Refactoring

- The ideas underlying refactoring are straightforward, but...
- The only way to really learn refactoring is to write (and rewrite and rewrite) lots of code
- If you are interested, check the online refactoring catalogue: http://www.refactoring.com/catalog

Lecture 4 Summary

- UML Notation: visibility, abstract, arrows (inheritance, composition)
- Concepts:
  - Encapsulation and Data Hiding
  - Interface and Implementation
  - Inheritance
  - Polymorphism
  - Composition
  - Refactoring